Contact-Grounded Policy: 분석 리포트

게시용 버전: 논문 그림은 라이선스(arXiv 비독점 배포) 때문에 제외했습니다. 원문 https://arxiv.org/abs/2603.05687 · arXiv:2603.05687v3 · RSS 2026 · Purdue, Meta Reality Labs Research, UW-Madison (Xu, Wang, Abbatematteo, Preechayasomboon, Chan, Colonnese, Memar) · 코드 미공개 · 분석일 2026-10-28 · 논문 그림은 개인 학습용으로만 사용(논문 라이선스가 CC BY 아님)

핵심 요약

1. **무엇을 하는가:** 다지 손 visuotactile diffusion 정책이 행동 대신 미래의 **실제 로봇 상태와 촉각(VAE 잠재)**을 예측한다. 학습된 contact-consistency mapping M(x, u) → a가 이를 compliance 제어기 목표로 바꾼다.
2. **M의 물리적 실체는 준정적 강성 역행렬이다:** 관절 PD에서 $a = x - Kp^{-1}(J^T f + \tau_g)$이다. 3-DoF 손가락 toy에서 해석식은 정확히 성립하고(오차 0.008 mrad), 순수 MLP는 20 mrad였다. 잔차형과 상태·촉각 결합이 필요한 이유도 이 식으로 설명된다.
3. **"일관되게 앞선다"는 점추정 기준이다:** 다섯 과제 모두 1위지만, 과제별로 유의한 것은 시뮬레이션 접시 닦기($p = 0.001$)뿐이다. 실제 로봇은 15회라 구분되지 않는다($p = 0.33, 0.43$).
4. **촉각 잡음 강건성은 과장되어 있다:** Fig. 13을 읽으면 잡음 50%에서 상태+촉각 매핑 오차(약 10.0)가 상태만 모델(10.64)과 거의 같다. 촉각 이득의 86%가 사라진 상태다.
5. **청크 실행 중에는 촉각 피드백이 없다:** M은 예측된 ($\hat{x}$, $\hat{u}$)에 적용되고, 8스텝 동안 관측 촉각은 목표에 반영되지 않는다. 손끝만 감지하면 근위 링크 접촉이 빠져 식별이 무너진다(toy에서 오차가 잔차 0 수준 근처까지).
6. **구현 가능성:** 공식 코드·데이터·시뮬레이터(Meta 내부 FEM + Unreal)가 없어 그대로는 재현할 수 없다. 구성 요소는 표준(Diffusion Policy + VAE + MLP)이라 재구현은 쉽고, 어려운 것은 (목표, 실제 상태, 촉각) 세 쌍 데이터와 촉각 시뮬레이션이다. 이 맥에서는 MuJoCo(CPU)로 매핑 실험을, PyTorch MPS로 작은 정책 학습을 해 볼 수 있다(둘 다 설치 필요).

01. 논문 분석: Contact-Grounded Policy (arXiv:2603.05687)

- 제목: *Contact-Grounded Policy: Dexterous Visuotactile Policy with Generative Contact Grounding*
- 저자: Zhengtong Xu, Yeping Wang, Ben Abbatematteo, Jom Preechayasomboon, Sonny Chan, Nick Colonnese, Amirhossein H. Memar (Purdue, Meta Reality Labs Research, UW-Madison). 1저자의 Meta Reality Labs 인턴십 연구입니다.
- 공개: arXiv v1 2026-03-05, v2 2026-03-09, **v3 2026-05-08**(분석 대상) · RSS 2026 · 프로젝트 https://contact-grounded-policy.github.io/
- 코드: **공개되지 않았습니다.** 프로젝트 페이지에는 논문과 영상(Google Drive) 링크만 있습니다. GitHub의 `contact-grounded-policy` 계정에는 웹페이지 저장소 하나만 있습니다(2026-10-28 확인).
- 라이선스: 논문은 **arXiv 비독점 배포 라이선스**(CC BY 아님). 프로젝트 웹페이지는 CC BY-SA 4.0입니다.
- 분석 범위: 본문 9쪽과 부록 A-E(총 14쪽), 프로젝트 페이지

---

1. Abstract (한 줄 요약과 주장)

한 줄 요약: 다지 로봇손의 visuotactile 모방학습에서 행동(목표 관절각)을 직접 예측하지 않습니다. 대신 **미래의 실제 로봇 상태와 촉각을 함께 예측**하고, 학습된 **contact-consistency mapping** $M_\varphi(x, u) \to a$로 이를 compliance 제어기가 추종할 목표 상태로 바꿉니다.

기여는 두 가지입니다(Sec. I).

1. **Contact grounding 정식화:** 촉각을 추가 관측이나 보조 예측 대상으로만 쓰지 않습니다. 고정된 센서·제어기 조합에서는 (실제 상태 x, 촉각 u, 목표 상태 a) 세 쌍이 접촉을 나타낸다고 보고, 예측한 $(\hat{x}, \hat{u})$를 실행 가능한 목표로 바꿉니다.
2. **잠재 공간 촉각 예측:** KL 정규화 VAE로 촉각을 압축하고(촉각 배열 748×3 → 32차원, Digit360 4개 이미지 → 80차원), 그 잠재 공간에서 상태와 함께 diffusion으로 생성합니다.

---

2. Method

2.1 전체 구조 (Fig. 3)

```
관측 O_t = {o_{t-1}, o_t}, o = (RGB 2시점 i, 촉각 u, 로봇 상태 x)          (T_o = 2)
        │  시각 인코더 + 촉각 인코더 + 상태 → FiLM 조건
        ▼
Diffusion U-Net π_θ (Diffusion Policy[4] 구조, DDIM 8스텝)
   출력 Y_t = [x_{t+1:t+16}, h_{t+1:t+16}]   (실제 상태 궤적 + 촉각 VAE 잠재 궤적, T = 16)
        ▼
Contact-consistency mapping (스텝별) â_{t+k} = M_φ(x̂_{t+k}, û_{t+k})     (Eq. 1, 잔차형: â = x̂ + Δ)
        ▼
compliance 제어기: 팔 = 작업공간 임피던스, 손 = 관절 PD.  8스텝 실행 후 재계획 (T_a = 8)
```

2.2 Contact grounding의 핵심 관찰 (Sec. III-B, Fig. 2)

관절마다 PD 제어기를 두면, 제어기는 목표 a와 실제 x의 차이를 토크로 바꾸는 가상 스프링-댐퍼가 됩니다. 자유 공간에서는 x가 a를 잘 따라가지만, 접촉이 생기면 물체가 손가락을 막아 **a와 x 사이에 간극**이 생기고, 그 결과가 촉각 u에 나타납니다. 저자들은 이 결합 관계를 데이터로 학습합니다.

```
a_t = M_φ(x_t, u_t)                                                    (Eq. 1)
```

접촉 위치나 접촉 모드를 명시적으로 모델링하지 않습니다. 대신 센서·제어기·embodiment에 묶인 암묵적 사상을 학습합니다. 이 식의 물리적 의미(준정적 평형에서 $a = x + Kp^{-1}(\cdots)$)는 `03` §1에서 유도했습니다.

2.3 정책 구성 (Sec. III-C-E)

- **분해의 이유:** 관측에서 목표로 바로 가지 않고, 먼저 "접촉이 어떻게 전개되어야 하는가"를 (상태, 촉각) 쌍의 궤적으로 생성합니다. 그다음 그것을 실현할 목표를 M_φ로 구합니다.
- **잔차 사상:** M_φ는 목표를 바로 회귀하지 않고 **실제 상태로부터의 offset**을 출력합니다. 학습이 안정되고 목표가 강건해진다고 설명합니다(Table III에서 오차 33-66% 감소).
- **촉각 VAE:** 촉각 배열은 taxel 축 방향 1D ResNet, Digit360 이미지는 센서별 가중치 공유 2D ResNet을 씁니다. KL 가중치는 1×10^{-5}-1×10^{-4}입니다(Table V).
- **조건 주입:** 시뮬레이션에서는 시각·촉각·상태 특징을 이어 붙여 FiLM으로 넣습니다. 실제 로봇에서는 센서별 촉각 특징을 cross-sensor self-attention으로 모은 뒤 FiLM으로 넣습니다.
- **M_φ의 두 설계:** 촉각 배열에서는 예측 잠재를 원래 촉각으로 복원한 뒤 다시 인코딩하고, 상태와 합쳐 MLP에 넣습니다(정확도 우선). Digit360에서는 이미지 복원이 비싸서 예측 잠재를 그대로 상태와 합쳐 MLP에 넣습니다(속도 우선).
- **인코더는 공유하지 않습니다.** diffusion 조건용, M_φ용, VAE용 인코더는 같은 구조이지만 따로 학습합니다.

2.4 시스템 (Sec. IV, Table V)

	시뮬레이션	실제 로봇
팔 / 손	UR5 + Tesollo DG-5F (5지 20 DoF)	Franka Panda + Allegro V5 (4지 16 DoF)
촉각	손가락 안쪽·손바닥 748 taxel × 3D 힘	손끝 Digit360 4개 (72×72 RGB)
시뮬레이터	실시간 FEM(축소 차수 모델, 데이터 기반 hyper-reduction) + Unreal Engine 플러그인	—
텔레옵	Meta Quest 3 손 추적 → RelaxedIK	OptiTrack + 계측 장갑 → RelaxedIK (새끼손가락 무시)
행동 / 상태 차원	29 (3 + rot6D 6 + 20) / 27 (3 + 쿼터니언 4 + 20)	25 / 23
촉각 잠재	32	80 (센서당 20)
시간 설정	5 Hz, T = 16, T_o = 2, T_a = 8, DDIM 8스텝	같음

---

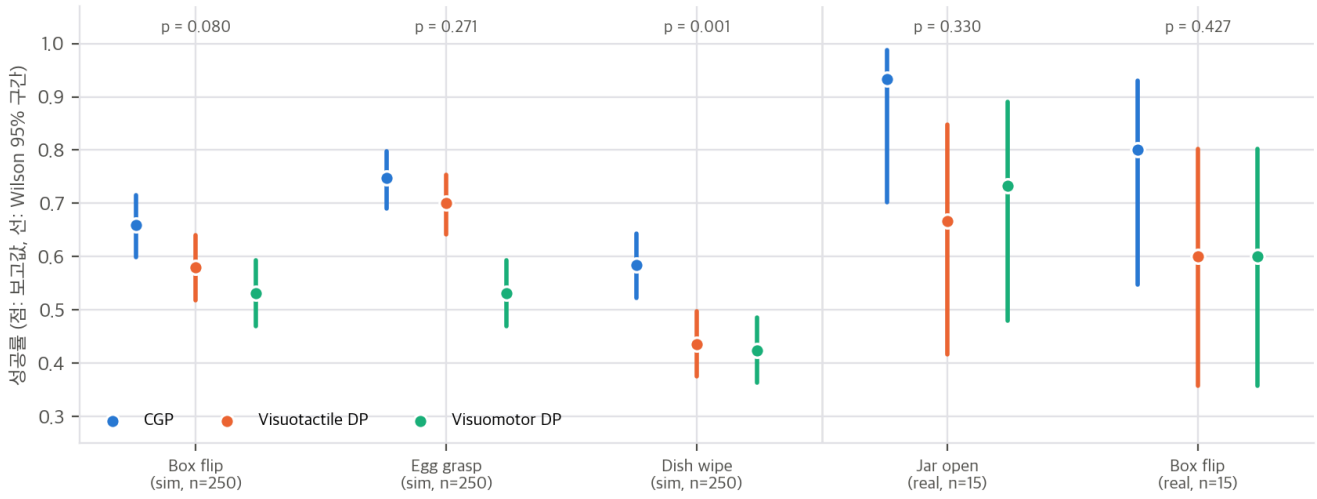
3. Experiments

3.1 주 결과 (Table II)

방법	Box Flipping (sim, 60 demos)	Egg Grasping (sim, 100)	Dish Wiping (sim, 100)	Jar Opening (real, 45)	Box Flipping (real, 90)
CGP	**66.0%** (165/250)	**74.8%** (187/250)	**58.4%** (146/250)	**93.3%** (14/15)	**80.0%** (12/15)
Visuotactile DP	58.0% (145)	70.0% (175)	43.6% (109)	66.7% (10)	60.0% (9)

방법	Box Flipping (sim, 60 demos)	Egg Grasping (sim, 100)	Dish Wiping (sim, 100)	Jar Opening (real, 45)	Box Flipping (real, 90)
Visuomotor DP	53.2% (133)	53.2% (133)	42.4% (106)	73.3% (11)	60.0% (9)

시뮬레이션은 마지막 5개 체크포인트에서 총 250회, 실제 로봇은 연속 15회 시행입니다. 괄호 안 성공 수는 우리가 역산했고, 모두 정수로 떨어집니다(`04` C1).



출처: Table II. p는 CGP vs 최강 기준선 Fisher 정확검정. sim 250회는 한 학습 실행의 체크포인트 5개 × 50회라 독립 시행이 아님.

우리 분석: Table II 성공률과 Wilson 95% 구간, CGP vs 최강 기준선 Fisher p.

해석 (재계산은 `04_claim_verification.md` 와 `analysis/table_check.py`):

- **"일관되게 앞선다"는 점추정으로는 맞습니다.** 다섯 과제 모두에서 CGP가 가장 높습니다.
- **과제별로 통계적으로 구분되는 것은 접시 닦기뿐입니다.** 최강 기준선(visuotactile DP)과의 Fisher 검정은 접시 닦기 $p = 0.001$, 박스 뒤집기 $p = 0.08$, 달걀 집기 $p = 0.27$입니다. 세 시뮬레이션 과제를 묶은 Mantel-Haenszel 검정은 $p = 2.4\times10^{-4}$입니다. 다만 250회는 한 학습 실행의 체크포인트 5개 × 50회라 서로 독립이 아니므로, 이 p값은 낙관적입니다. 학습 seed 반복은 보고되지 않았습니다.
- **실제 로봇 결과는 15회라 구분되지 않습니다.** 병뚜껑 열기 14/15 vs 11/15($p = 0.33$), 박스 뒤집기 12/15 vs 9/15($p = 0.43$)입니다. 묶어도 $p = 0.13$입니다. 15회 대 15회에서 $p < 0.05$가 되려면 14/15 대 8/15 정도, 즉 40%p에 가까운 차이가 필요합니다.
- **촉각을 관측으로만 넣는 것의 효과가 작습니다.** visuomotor → visuotactile DP는 달걀 집기에서만 크게 오르고(+16.8%p), 병뚜껑 열기에서는 오히려 떨어집니다(73.3% → 66.7%). 이 결과는 "촉각을 관측에만 넣는 것으로는 부족하다"는 논문의 동기와 맞습니다.

3.2 Contact-consistency mapping 단독 평가 (Table III, Fig. 6, Fig. 13)

시뮬레이션에서 11개 물체를 150개 에피소드(4,114프레임, 에피소드당 평균 27프레임) 동안 텔레옵으로 잡고, 에피소드 단위 1:1로 학습·검증을 나눴습니다. (x, u) → a 예측의 관절각 MAE입니다($\times10^{-3}$ rad, 3 seed 평균 ± 표준편차).

입력	인코더	절대	잔차
상태 + 촉각	ResNet1D	8.80 ± 0.24	**5.94 ± 0.20**
	MLP / Transformer	12.50 / 14.39	8.33 / 9.58
상태만	—	16.05	10.64

입력	인코더	절대	잔차
촉각만	ResNet1D	35.93	12.15

- 상태와 촉각이 **둘 다 필요합니다.** 잔차 기준으로 상태+촉각은 상태만보다 44%, 촉각만보다 51% 낮습니다(3 seed Welch $p < 10^{-3}$).
- **잔차형이 일관되게 낫습니다.** 상태가 들어간 경우 33%, 촉각만일 때 66% 줄어듭니다. 촉각만+잔차가 크게 좋아지는 이유는 단순합니다. 잔차형에서는 실제 상태 x를 네트워크 밖에서 더해 주므로, 촉각만 쓰는 모델도 사실상 x를 받습니다.
- 최고 MAE 5.94×10^{-3} rad는 **0.34°**입니다. 그러나 비교 기준인 **"a = x"(잔차 0) 오차와 접촉 중 평균 간극 |a − x|가 보고되지 않았습니다.** 이 수치가 얼마나 좋은지 판단하기 어렵습니다(`04` C3).
- **촉각 잡음 강건성 (부록 E, Fig. 13):** 저자들은 "넓은 잡음 범위에서 안정적"이라고 썼습니다. 그림에서 읽으면 상태+촉각·잔차 오차는 잡음 0%에서 약 6.0, 50%에서 약 10.0으로 67% 오릅니다. 50%에서의 10.0은 상태만 모델(10.64)과 거의 같아서, **촉각이 주던 이득의 약 86%가 사라진 상태**입니다. 직선으로 늘리면 잡음 약 58%에서 상태만 모델과 같아집니다. 실제 배포에서 diffusion이 예측한 촉각의 오차가 데이터셋 표준편차의 몇 %인지는 보고되지 않았습니다.

3.3 촉각 압축 (Table IV, Fig. 7, 부록 C)

- KL 정규화를 빼면 복원 MAE는 낮아지지만 잠재 KL은 2-6배 커지고, 정책 성공률은 떨어집니다. 그림에서 읽으면 박스 66% → 약 62%, 달걀 75% → 약 64%입니다. 이 ablation은 세 시뮬레이션 과제 중 두 과제에서만 보고되었습니다. 박스의 4%p 차이(250회 기준 $p \approx 0.40$)는 잡음 수준이고, 달걀의 11%p($p \approx 0.009$)는 의미가 있습니다.
- **MAE와 Active MAE의 비로 활성 taxel 비율을 거꾸로 추정할 수 있습니다.** MAE = f·(Active MAE) + (1 − f)·(비활성 오차)이고 비활성 오차는 0 이상이므로 $f \le$ MAE / Active MAE입니다. 계산하면 박스 ≤ 10%, 달걀 ≤ 12%, 접시 ≤ 23%입니다. 748개 taxel 가운데 한 시점에 힘이 걸리는 것은 많아야 75-170개라는 뜻으로, 촉각 배열이 매우 희소합니다.
- Digit360 압축률은 센서당 15,552값(72×72×3) → 20차원으로 약 780배입니다. PSNR에서 거꾸로 구한 RMSE는 MAE의 1.8-2.1배로, 가우시안 오차(1.25배)보다 꼬리가 무겁습니다. 복원 오차가 접촉 영역에 몰려 있다는 뜻으로 읽힙니다.

3.4 추론 시간 (Fig. 8)

A100에서 DDIM 8스텝 기준으로, 그림에서 읽은 총 시간은 다음과 같습니다.

	Visuomotor DP	Visuotactile DP	CGP
촉각 배열	약 93 ms	약 97 ms	약 112 ms (mapping 약 12 ms)
Digit360	약 96 ms	약 133 ms	약 134 ms (mapping 약 3 ms)

diffusion 출력 차원은 CGP가 DP보다 2배(시뮬레이션 16×(27+32) = 944 vs 464), 실제 로봇에서는 4배(1,648 vs 400)입니다. 그래도 U-Net 시간은 거의 같습니다. 출력 차원보다 U-Net 폭이 비용을 좌우하기 때문으로 보입니다.

---

4. Limitations

4.1 저자가 밝힌 한계 (Sec. VI)

M_φ는 센서 종류와 제어기 설정에 묶여 있어 그대로 옮겨 쓸 수 없습니다. sim-to-real 전이를 다루지 않았습니다(시뮬레이션과 실제를 따로 평가). 과제마다 따로 학습하는 단일 과제 프로토콜입니다. 향후 과제로 센서·제어기 간 공동 학습, 제어기 이득과 로봇 물리 파라미터를 조건으로 넣는 방법, 과제 간 공동 학습을 들었습니다.

4.2 분석하면서 추가로 발견한 한계와 의문점

1. **청크 실행 중에는 촉각 피드백이 정책에 들어가지 않습니다.** M_φ는 관측 촉각이 아니라 **예측된** ($\hat{x}$, $\hat{u}$)에 적용됩니다. 8스텝을 실행하는 동안 실제 촉각이 예측과 달라져도 목표는 바뀌지 않고, 반응은 compliance 제어기의 수동적 순응에 맡겨집니다. "5 Hz"가 행동 스텝 주기라면 재계획은 1.6초마다입니다. 추론 주기라면 행동 스텝은 25 ms입니다. 논문은 둘 중 어느 쪽인지 밝히지 않았습니다(`04` C8).
2. **M_φ의 물리적 실체는 준정적 강성 역행렬입니다**(`03` §1). 관절 PD에서 평형은 $a = x + Kp^{-1}(g(x) - \Sigma J^{T}f)$입니다. 접촉력 f와 접촉점 Jacobian을 알면 사상은 해석적으로 정해집니다. 시뮬레이션은 taxel마다 3D 힘을 주므로, 해석식이나 "해석식 + 학습 잔차" 기준선이 가능했지만 비교하지 않았습니다. 저자들이 향후 과제로 든 "제어기 이득을 조건으로" 넣는 방향도, 해석식에서는 $\Delta \propto Kp^{-1}$이라 구조가 이미 정해져 있습니다.
3. **실제 상태 x에 속도가 없습니다.** x는 손목 자세와 관절각뿐입니다. PD의 감쇠 항($Kd \cdot \dot{x}$)과 이동 중 목표가 실제 상태를 앞서는 성분은 (x, u)로 설명되지 않아 오차 하한으로 남습니다(`03` §3, toy M4).
4. **손끝 센서(Digit360)에서는 식별이 불완전합니다.** 손가락 마디나 손바닥 접촉은 감지되지 않고, 비전 기반 촉각은 보정된 힘이 아니라 변형 이미지입니다. 실제 로봇 실험은 손끝 중심 과제(병뚜껑, 박스 뒤집기)로 골랐지만, 손 전체 접촉이 필요한 과제에서 어떻게 되는지는 실험되지 않았습니다. M_φ를 단독으로 평가한 Table III도 시뮬레이션 촉각 배열에서만 했습니다.
5. **통계 정보가 부족합니다.** 학습 seed 반복이 없고, 실제 로봇은 15회입니다. 시뮬레이션 250회도 독립 시행이 아닙니다(§3.1).
6. **재현 경로가 막혀 있습니다.** 코드가 공개되지 않았고, 시뮬레이션은 Meta 내부의 FEM 솔버와 Unreal Engine 플러그인입니다. 공개 시뮬레이터로 같은 과제를 다시 만들어야 합니다(`04` §4).
7. **정책 출력 차원이 커집니다.** 행동 대신 상태와 촉각 잠재를 예측하므로 diffusion 출력이 2-4배입니다. 손 전체 촉각을 더 촘촘히 하거나 예측 구간을 늘리면 이 비용이 커집니다. 잠재 차원을 32와 80으로 작게 잡은 이유이기도 합니다.

03. Contact-consistency mapping의 물리적 의미와 toy 검증

재계산 스크립트는 `analysis/mapping_toy.py` (결과 `mapping_toy_results.json`, CPU 약 2분)와 `analysis/table_check.py` 입니다. toy는 논문 Fig. 2와 같은 3-DoF 평면 손가락을 관절 PD로 제어하는 설정입니다. 논문 수치를 재현하려는 것이 아니라 Eq. 1이 무엇을 학습하는지 확인하는 것이 목적입니다.

1. Eq. 1 $M_\varphi(x, u) \rightarrow a$는 무엇을 학습하는가

1.1 관절 PD 손가락의 준정적 평형

손 관절은 PD 제어기 $\tau_m = Kp(a - q) - Kd\,\dot{q}$로 구동됩니다(Sec. III-A). 관성을 무시할 만큼 느린 움직임(준정적)이라면 관절 토크 평형은 다음과 같습니다.

$$Kp\,(a - q) - Kd\,\dot{q} + \Sigma_k\, J_k(q)^\top f_k + \Sigma_i\, J_{c,i}(q)^\top m_i\, g = 0$$

- f_k: 접촉점 k에서 물체가 손가락에 가하는 힘, J_k: 그 점의 Jacobian
- $J_{c,i}$, $m_i\, g$: 링크 i 질량중심의 Jacobian과 중력

이를 목표 a에 대해 풀면 다음 식이 됩니다.

$$a = \underbrace{q}_{\text{실제}} - \underbrace{Kp^{-1}\left[\,\Sigma_k\, J_k(q)^\top f_k + \tau_g(q)\,\right]}_{\text{접촉·중력이 만드는 간극 } (x,\ u\text{의 함수})} + \underbrace{Kp^{-1}\,Kd\,\dot{q}}_{\text{속도 항 } (x\text{에 없음})}$$

해석:

- 논문의 "잔차 사상"($a = x + \Delta$)은 이 구조와 정확히 맞습니다. 잔차 Δ는 **강성 역행렬 × 외력 토크**입니다.
- Δ를 정하려면 접촉점 위치와 힘(촉각 u)뿐 아니라 Jacobian, 즉 자세(상태 x)도 필요합니다. 그래서 "상태와 촉각이 둘 다 필요하다"(Table III)는 결과가 나옵니다.
- 접촉이 없으면 $\Delta = -Kp^{-1}\tau_g(q)$뿐입니다. 부록 Fig. 12가 "접촉 전 작은 간극은 중력과 마찰에 의한 정상상태 오차"라고 한 것과 같습니다.
- 팔 쪽 작업공간 임피던스 $F = K_x(a_x - x) - D\,\dot{x}$에서도 똑같이 $a_x = x + K_x^{-1}F_{ext}$입니다. Fig. 5에서 접시 닦기 때 엔드이펙터 목표-실제 간극이 커지는 것은 스펀지를 누르는 힘이 K_x^{-1}만큼 목표를 밀어 넣은 결과입니다.
- 저자들이 향후 과제로 든 "제어기 이득을 조건으로" 넣는 방향(Sec. VI)도, 이 식에서는 $\Delta \propto Kp^{-1}$로 구조가 이미 정해져 있습니다.

1.2 무엇이 식별을 깨는가

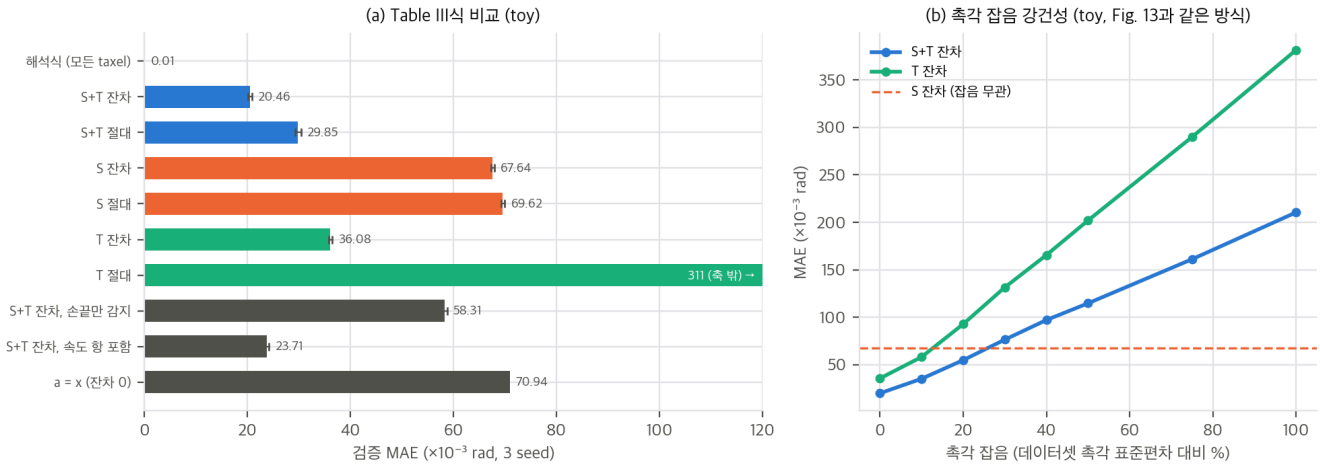
위 식이 정확하려면 세 조건이 필요합니다. 실제 시스템에서는 셋 다 깨질 수 있습니다.

1. **모든 접촉이 감지되어야 합니다.** 손끝 Digit360만 있으면 손가락 마디나 손바닥 접촉의 f_k가 빠집니다.
2. **촉각이 힘을 줘야 합니다.** 시뮬레이션 촉각 배열은 taxel마다 3D 힘을 줍니다. 비전 기반 촉각은 변형 이미지라서 힘과의 관계도 학습해야 합니다.
3. **속도 항이 작아야 합니다.** x에는 관절 속도가 없으므로 $Kp^{-1}Kd\,\dot{q}$는 (x, u)로 설명되지 않는 오차 하한이 됩니다. 텔레옵 데이터에서는 이동 중 목표가 실제 상태를 앞서는 성분도 여기에 섞입니다.

2. toy 설정

- 3링크 평면 손가락(50 / 35 / 25 mm), 굴곡 방향으로 말림. Kp = [1.5, 1.0, 0.6] N·m/rad, Kd = [0.03, 0.02, 0.012], 링크 질량 [30, 20, 12] g
- 손바닥 쪽 taxel 링크당 6개(총 18개), taxel마다 법선력 측정
- 무작위 목표 a와 무작위 원형 물체(반지름 12-35 mm)를 뽑고, 에너지 ½(q−a)ᵀKp(q−a) + penalty 접촉 + 중력 위치에너지를 최소화해 평형 q를 구함 → (q, u, a) 6,000개. 접촉 표본 73%
- 1:1 학습·검증 분할(논문과 같은 비율), 2층 ReLU MLP(폭 128), 3 seed

3. 결과 (`mapping_toy_results.json`)



3-DoF 평면 손가락, 관절 PD, 원형 물체 penalty 접촉, 준정적 평형. analysis/mapping_toy.py. 수치는 논문과 직접 비교 대상이 아니라 경향 확인용.

모델	검증 MAE (×10⁻³ rad)	논문 Table III 대응값
해석식 (Kp, 기구학, 모든 taxel 힘을 앎)	**0.008**	없음
MLP 상태 + 촉각, 잔차	20.5 ± 0.4	5.94
MLP 상태 + 촉각, 절대	29.9 ± 0.6	8.80
MLP 상태만, 잔차	67.6 ± 0.3	10.64
MLP 촉각만, 잔차	36.1 ± 0.3	12.15
MLP 촉각만, 절대	311 ± 0.6	35.93
a = x (잔차 0)	70.9	**보고 안 됨**
MLP 상태 + 촉각, 잔차, **손끝 taxel만 감지**	58.3 ± 0.5	실제 로봇(Digit360)에 해당
해석식, 손끝 taxel만 감지	51.1	
MLP 상태 + 촉각, 잔차, **속도 항 포함** ($\dot{q}$ ~ N(0, 0.4 rad/s))	23.7 ± 0.5	하한 6.4

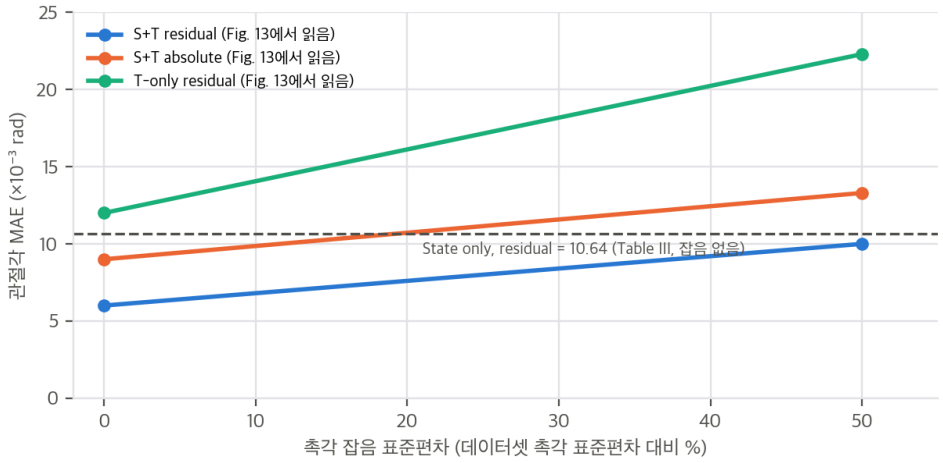
확인된 것:

1. **Eq. 1의 해석식이 정확히 성립합니다**(오차 0.008, 수치 최적화 허용오차 수준). 이 사상은 "데이터로만 배울 수 있는 암묵적 관계"가 아니라 **기구학과 강성으로 정해지는 관계**입니다. 학습이 필요한 이유는 실제 시스템에서 강성, 마찰, 센서 보정, 감지되지 않는 접촉을 정확히 모르기 때문입니다.
2. **잔차형과 상태·촉각 결합의 이득은 논문과 같은 방향입니다.** 잔차가 절대보다 낫고(20.5 vs 29.9), 촉각만·절대는 자세를 알 수 없어 크게 실패합니다(311).

3. **상태만 모델의 가치는 데이터 분포에 달려 있습니다.** toy에서는 물체 위치와 크기가 무작위라 자세만으로 힘을 알 수 없고, 상태만 모델(67.6)이 잔차 0 기준선(70.9)과 거의 같습니다. 논문에서는 상태만 모델(10.64)이 촉각만(12.15)보다 낫습니다. 파지 데이터에서는 자세가 "얼마나 쥐고 있는가"를 상당 부분 알려 준다는 뜻으로 읽힙니다. 같은 이유로 논문에 잔차 0 기준선이 없으면 5.94가 얼마나 좋은 값인지 판단할 수 없습니다.
4. **학습된 MLP는 해석식보다 2,000배 이상 부정확합니다**(20.5 vs 0.008). 학습 표본 3,000개로는 $J^{\mathsf{T}}f$라는 곱 구조를 배우기 어렵습니다. 기구학을 아는 경우 "해석식 + 학습 잔차" 구조가 데이터 효율 면에서 훨씬 유리할 것입니다.
5. **손끝만 감지하면 식별이 크게 무너집니다.** 표본의 59%에서 감지되지 않는 근위 링크 접촉이 있고, MLP 오차가 58.3으로 잔차 0 기준선(70.9)에 가까워집니다. 해석식도 51.1입니다. 실제 로봇 실험이 손끝 접촉 위주 과제(병뚜껑, 박스 뒤집기)였던 것은 이 한계와 맞물립니다.
6. **속도 항은 하한을 만듭니다.** 하한($K_p^{-1}K_d|\dot{q}|$의 평균 6.4)이 생기고, 오차가 20.5 → 23.7로 늘어납니다.

논문과 다른 것 (촉각 잡음, M5): toy의 상태+촉각·잔차 모델은 데이터셋 촉각 표준편차의 10%만 잡음을 넣어도 오차가 20 → 35로 오르고, 약 25%에서 상태만 모델보다 나빠집니다. 논문 Fig. 13은 50%에서도 +67%에 그칩니다. 차이의 이유로는 세 가지를 생각할 수 있습니다. toy의 taxel 신호는 대부분 0이어서, 잡음이 "없는 접촉"을 만들어 냅니다. toy는 잡음 없이 학습했습니다. 논문의 ResNet1D 인코더는 공간적으로 매끄러운 특징을 써서 독립 잡음을 평균화합니다. 어느 쪽이든 강건성은 **잡음을 무엇에 상대적으로 정의하는가**와 학습 방식에 크게 좌우되므로, 논문의 "넓은 범위에서 안정적"이라는 표현을 일반화하기는 어렵습니다.

4. 논문 Fig. 13 다시 읽기



양 끝점만 표시. 잡음 50%에서 State+Tactile 잔차(10.0)는 상태만 모델(10.64)과 거의 같다.

논문 자체 수치로도, 잡음 50%에서 상태+촉각·잔차 MAE(약 10.0)는 Table III의 상태만 모델(10.64)과 거의 같습니다. 촉각이 주던 이득(10.64 − 5.94 = 4.70) 가운데 남는 것은 약 0.64(14%)입니다. "오차가 안정적"이라기보다 **"촉각이 무의미해질 때까지 점진적으로 나빠진다"**가 정확한 표현입니다. 배포 시 diffusion이 예측한 촉각의 오차 크기가 보고되지 않았으므로, 실제 정책에서 이 곡선의 어느 지점에 있는지는 알 수 없습니다.

5. 실행 시간과 피드백 구조 (`04` C8)

CGP는 매 추론마다 16스텝의 $(\hat{x}, \hat{u})$를 만들고 8스텝을 실행합니다. M_φ는 예측값에 적용되므로, 실행 중인 8스텝 동안 실제 촉각은 목표에 반영되지 않습니다. "5 Hz"를 어떻게 읽느냐에 따라 시간 규모가 달라집니다.

해석	행동 스텝	재계획 주기	예측 구간	A100 추론(최대 약 134 ms)
5 Hz = 행동 스텝 주기	0.2 s	1.6 s	3.2 s	재계획 주기의 8%
5 Hz = 추론 주기	25 ms	0.2 s	0.4 s	주기의 67%

어느 쪽이든 미끄러짐 같은 빠른 사건에는 제어기의 수동적 순응이 먼저 반응하고, 정책은 다음 재계획에서야 촉각을 봅니다. 청크 안에서 관측 촉각과 예측 촉각의 차이(u_obs − û)로 목표를 보정하면 이 간극을 줄일 수 있습니다.

04. 주장 검증: 표·그림 수치 재계산, 구현 가능성

- **대상:** arXiv:2603.05687v3 본문 Table II-VI, Fig. 7, 8, 13, 프로젝트 페이지
- **범위:** 공식 코드가 공개되지 않아서(2026-10-28 확인) 코드 대조는 할 수 없습니다. 표 수치를 재계산하고, 그림 값은 눈으로 읽어 근사했으며(±1 눈금), Eq. 1은 CPU toy로 검증했습니다.
- **스크립트:** `analysis/table_check.py` → `table_check_results.json`, `analysis/mapping_toy.py` → `mapping_toy_results.json`

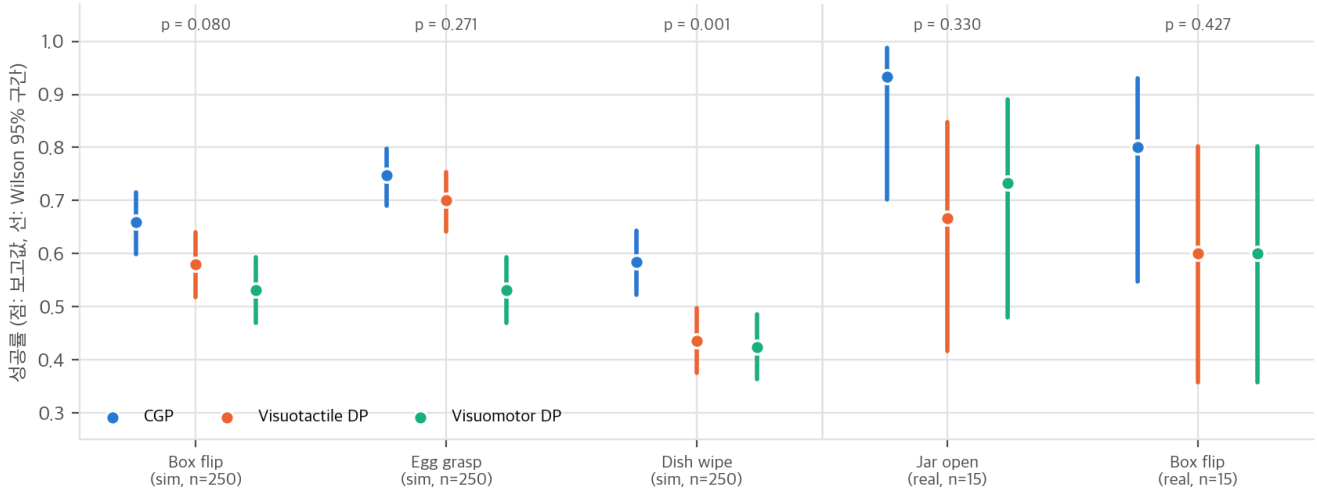
1. 주장별 판정

#	주장 (출처)	재계산·근거	판정
C1	Table II 수치 (sim 250회, real 15회)	모든 성공률 × 시행 수가 정수(예: 165, 145, 133 / 14, 10, 11)	✅ 내부 일관
C2	"CGP가 기준선을 일관되게 앞선다" (Sec. V-A)	점추정은 다섯 과제 모두 1위. 최강 기준선 대비 Fisher p: 박스(sim) 0.080, 달걀 0.271, 접시 **0.001**, 병뚜껑 0.330, 박스(real) 0.427. sim 묶음 MH p = 2.4×10^{-4}(비독립 시행이라 낙관적), real 묶음 p = 0.13	⚠️ sim 일부만 유의, real은 표본 부족
C3	상태와 촉각이 모두 필요하고 잔차형이 낫다 (Table III)	잔차 기준 상태+촉각은 상태만보다 44%, 촉각만보다 51% 낮음(3 seed Welch p = 3×10^{-4}, 3×10^{-6}). 잔차형은 33-66% 감소	✅ 단, a = x 기준선과 평균 간극이 없음
C4	Eq. 1은 접촉 위치·모드나 동역학을 명시하지 않는 암묵적 사상 (Sec. III-B)	toy에서 해석식 $a = q - Kp^{-1}(J^{T}f + \tau_g)$가 오차 0.008×10^{-3} rad로 정확히 성립. MLP는 20.5	⚠️ 사상의 구조는 해석적으로 정해짐 (`03` §1)
C5	매핑은 촉각 잡음에 강건하다 (부록 E, Fig. 13)	그림에서 읽으면 상태+촉각·잔차 6.0 → 10.0(잡음 50%, +67%). 50%에서 상태만 모델(10.64)과 거의 같아 촉각 이득의 86%가 사라짐. 직선 외삽 시 약 58%에서 같아짐	⚠️ "안정적"은 과장
C6	KL 정규화가 정책 성능을 높인다 (Fig. 7)	그림에서 읽으면 박스 66 vs 약 62($p \approx 0.40$), 달걀 75 vs 약 64($p \approx 0.009$). 접시 닦기는 보고 안 됨	⚠️ 두 과제 중 하나만 유의
C7	Table IV 복원 지표	MAE / Active MAE ≤ 활성 taxel 비율 → 박스 ≤ 10%, 달걀 ≤ 12%, 접시 ≤ 23%. KL 제거 시 KL 손실 1.9-6.1배. Fig. 7의 MAE·KL 막대가 Table IV ResNet1D 행과 일치	✅ 일관. 촉각 배열은 매우 희소
C8	Table VI (Digit360)	PSNR로 구한 RMSE가 MAE의 1.8-2.1배(가우시안이면 1.25배) → 오차가 접촉 영역에 몰림. 압축 15,552 → 20 (약 780배)	✅ 일관
C9	Table V 차원	29 = 3 + 6 + 20, 27 = 3 + 4 + 20, 25 = 3 + 6 + 16, 23 = 3 + 4 + 16	✅
C10	추론 시간이 기준선과 비슷하다 (Fig. 8)	그림에서 읽은 총 시간: 촉각 배열 93 / 97 / 112 ms, Digit360 96 / 133 / 134 ms. mapping은 12 ms(배열, 복원·재인코딩 포함)와 3 ms. diffusion 출력 차원은 CGP가 2배(sim), 4배(real)	✅ 촉각 배열에서는 +15%
C11	실행 시간 구조	"5 Hz"가 행동 스텝이면 재계획 1.6 s, 추론 주기이면 행동 스텝 25 ms. 어느 쪽이든 청크 실행 중에는 관측 촉각이 목표에 반영되지 않음	논문에 명시 없음
C12	평가 프로토콜	sim: 한 학습 실행의 마지막 5개 체크포인트 × 50회, 학습 seed 반복 없음. real: 연속 15회. 15 대 15에서 $p < 0.05$가 되려면 대략 14/15 대 8/15(차이 40%p)가 필요	⚠️ 통계 검정력 낮음
C13	매핑 데이터 규모	150 에피소드 / 4,114 프레임(에피소드당 27프레임), 1:1 분할이라 학습은 75 에피소드	참고

2. 보고되지 않아서 확인할 수 없는 것

- 공식 코드, 학습된 가중치, 데이터셋, 시뮬레이터(Meta 내부 FEM + Unreal Engine 플러그인)
- 네트워크 크기(U-Net 채널, VAE 폭), 학습 epoch과 시간, 학습 하드웨어. 추론 시간만 A100으로 보고
- M_φ의 학습 데이터와 시점. 정책 데모로 학습했는지, 별도 파지 데이터로 학습했는지 본문에 명확하지 않음
- 손 관절 PD 이득과 팔 임피던스 이득 값
- 실제 배포 중 diffusion이 예측한 촉각의 오차 크기(Fig. 13 곡선의 어느 지점인지)
- Table III의 $a = x$ 기준선과 접촉 중 평균 $|a - x|$

3. 재계산 결과 요약 그림



출처: Table II. p는 CGP vs 최강 기준선 Fisher 정확검정. sim 250회는 한 학습 실행의 체크포인트 5개 × 50회라 독립 시행이 아님.

4. 구현 가능성 판단

4.1 공식 코드를 그대로 돌리는 경우

불가능합니다. 2026-10-28 기준 코드, 가중치, 데이터가 공개되지 않았습니다. 프로젝트 페이지의 링크는 논문과 영상뿐이고, GitHub `contact-grounded-policy` 계정에는 웹페이지 저장소만 있습니다. 1저자의 GitHub에도 이전 연구(UniT, DiffOG, LeTO 등) 저장소만 있습니다. 시뮬레이션 실험은 Meta 내부 FEM 솔버와 Unreal Engine 플러그인을 쓰므로, 코드가 공개되더라도 시뮬레이션 부분은 그대로 재현하기 어려울 가능성이 큽니다.

4.2 이 맥에서 할 수 있는 것 (Apple M4 Pro, CUDA 없음)

단계	가능 여부	근거
표·그림 수치 재계산, Eq. 1 해석과 toy 검증	✅ 완료	`table_check.py`, `mapping_toy.py` (numpy, CPU 약 2분)
공개 시뮬레이터로 매핑 실험(Table III) 재현	✅ 가능할 것으로 보임	MuJoCo는 CPU에서 돌고, 접촉력과 touch 센서를 줍니다. 손 모델과 스크립트 파지로 (x, u, a) 데이터를 만들 수 있습니다. MuJoCo 설치가 필요합니다
CGP 정책 학습(작은 규모)	⚠️ 조건부	구조는 Diffusion Policy + VAE + MLP라 큰 모델이 아닙니다. PyTorch MPS로 작은 저차원 과제는 학습할 수 있을 것으로 보이지만, torch 설치가 필요하고 MPS에서의 학습 속도는 확인하지 않았습니다. 데이터는 직접 만들어야 합니다

단계	가능 여부	근거
논문 과제 재현(FEM 시뮬레이션, Allegro + Digit360 실물)	❌	시뮬레이터 비공개, 하드웨어 없음

4.3 아이디어를 다시 구현하는 경우

구성 요소	난이도	메모
Diffusion 정책 뼈대	낮음	공개된 Diffusion Policy 코드에서 출력만 바꿉니다. 행동 대신 [실제 상태, 촉각 잠재]를 예측합니다
촉각 VAE	낮음	1D ResNet(배열) 또는 2D ResNet(이미지), KL 가중치 1e-5-1e-4(Table V)
Contact-consistency mapping	낮음	잔차 MLP. 기구학과 강성을 알면 해석식 $\Delta = -Kp^{-1}(J^{T}f + \tau_g)$를 기본값으로 두고 잔차만 학습하는 편이 데이터 효율이 좋습니다(`03` §3)
학습 데이터 (x, u, a 세 쌍)	**높음**	목표(명령)와 실제 상태, 촉각을 모두 기록한 텔레옵 데이터가 필요합니다. 대부분의 공개 데이터셋은 명령과 실제 상태를 함께 저장하지 않습니다. compliance 제어기 위에서 직접 수집해야 합니다
촉각이 있는 시뮬레이션	중간-높음	손 전체 촉각 배열이 필요합니다. 공개 시뮬레이터로 만들 수 있지만, 연성체(스펀지)와 깨지는 달걀 같은 과제 자산은 새로 만들어야 합니다
실제 로봇	높음	다지 손 + 손끝 촉각 + compliance 제어기 + 손 추적 텔레옵

요약: 공식 코드로 재현하는 경로는 없습니다. 알고리즘 자체는 표준 구성 요소(Diffusion Policy, VAE, MLP)라 다시 구현하기 쉽습니다. 어려운 부분은 (실제 상태, 촉각, 목표) 세 쌍을 담은 데이터와 촉각 시뮬레이션입니다. 이 맥에서는 MuJoCo로 매핑 부분을 먼저 검증하고, 필요하면 작은 정책 학습을 MPS로 시도하는 순서가 현실적입니다. 둘 다 패키지 설치가 필요합니다.

02. 선행연구와 계보

번호 [n]은 논문의 참고문헌 번호이고, arXiv ID는 논문 참고문헌 목록에 적힌 값을 그대로 옮겼습니다. 이 문서의 각 논문을 별도로 열어 내용을 검증하지는 않았습니다.

1. 계보도

```
[행동 생성] Diffusion Policy [4] (RSS 2023) — 관측 → 행동 청크, U-Net, DDIM, FiLM 조건
        │
        ├─ [촉각을 관측으로 추가] 3D-ViTac [13], PolyTouch [41], Reactive DP [34], FACTR [19],
ViTacFormer [9]
        │     촉각을 입력 특징이나 보조 예측 목표로 사용 (Table I의 "executable contact modeling 없음")
        │
        ├─ [촉각·힘을 예측 목표로] ViTacFormer [9], Unified Multimodal Diffusion Forcing [14],
MLA [20]
        │     미래 촉각·힘을 예측해 표현을 다듬지만, 예측한 접촉을 제어 목표와 직접 잇지 않음
        │
        ├─ [제어 가능한 접촉 표현: 단일 힘 벡터] Adaptive Compliance Policy [12], Compliant Residual
DAgger [31], UMI-FT [5]
        │     팔 엔드이펙터 힘 하나 또는 그리퍼 손가락당 힘 하나 → 임피던스 목표
        │
        └─ [제어 가능한 접촉 표현: 손끝 힘] KineDex [39], DexForce [3]
              다지 손이지만 손끝마다 sparse한 힘
                    ▼
  ★ CGP: 손 전체의 분포 접촉을 (실제 상태, 촉각) 궤적으로 생성하고, 학습된 사상으로 compliance 목표로 변환
```

차별점 요약 (Table I): 제어기가 실행할 수 있는 접촉 모델을 두면서, 다지 손과 **분포 접촉**(손 전체 다중 패치)까지 다루는 것은 CGP뿐이라고 주장합니다. 차이는 접촉을 힘 벡터 몇 개로 매개화하지 않고 촉각 신호 자체를 예측한다는 점입니다.

2. 구성 요소별 출처

역할	출처	이 논문에서의 쓰임
조건부 diffusion 정책	Diffusion Policy [4]	U-Net denoiser, T_o = 2 / T = 16 / T_a = 8 기본값 그대로
학습·샘플링	DDPM [11], DDIM [26]	노이즈 예측 학습, 8스텝 DDIM 추론
잠재 공간 생성	Latent Diffusion [24], VAE [16]	촉각을 KL 정규화 VAE 잠재로 압축한 뒤 생성
조건 주입	FiLM [21]	시각·촉각·상태 특징을 U-Net 전체에 주입
회전 표현	rot6D [42]	목표 회전 6차원
팔 제어	deoxys [28], VIOLA [43]의 작업공간 임피던스	팔 목표 = 임피던스 setpoint
텔레옵 IK	RelaxedIK [23], RangedIK [29] (2저자 Yeping Wang의 선행연구)	사람 손끝 → 로봇 손가락 관절
손 추적	UmeTrack 8, 마커 기반 손 추적 [7](OptiTrack + 장갑)	시뮬레이션·실제 텔레옵
촉각 센서	Digit360 [17] (Meta)	실제 손끝 4개, 72×72 RGB
시뮬레이터	축소 차수 FEM [27, 2, 44, 25] (Meta Reality Labs 계열) + Unreal Engine	강체·연성체(스펀지) 접촉, 748 taxel 촉각 배열
촉각 표현 선행연구	UniT [33] (1저자의 선행연구, RA-L 2025)	촉각 표현 학습 계보

3. 이 저장소의 다른 논문과의 관계

논문	관계
GPT-as-Tactile-Policy (`gpt6_astra_tactile/`)	둘 다 촉각 정책이지만 촉각을 쓰는 위치가 다릅니다. 그쪽은 촉각을 정책의 입력(판단 근거)으로 쓰고, CGP는 촉각을 **예측 대상이자 제어 목표의 근거**로 씁니다. 그쪽 UniVTAC 2×2 재현 실험에서는 촉각이 Task Skill(촉각을 쓰는 규칙)과 함께 있을 때만 유의했습니다(+50%p, Skill이 없으면 +15%p로 비유의). CGP Table II에서 촉각을 관측으로만 넣은 visuotactile DP의 이득이 작거나 음수인 것(병뚜껑 열기 73.3% → 66.7%)과 같은 방향의 관찰입니다. 두 결과 모두 촉각을 "어떻게 쓰는지"가 구조로 주어질 때 효과가 난다는 쪽을 가리킵니다.
DexScrew (`dexscrew/`)	텔레옵으로 다지 손 촉각 데이터를 모아 BC를 학습한다는 점이 같습니다. DexScrew는 어려운 손가락 동작을 RL 기술로 맡기고, CGP는 compliance 제어기와 목표-실제 간극으로 접촉을 다룹니다. 두 방법은 결합할 수 있습니다.
FlashDexRetarget (`flashdexretarget/`)	FlashDexRetarget 정책은 링크별 접촉 wrench를 privileged 관측으로 씁니다. CGP의 M_φ는 촉각에서 그와 같은 접촉 효과(목표-실제 간극)를 복원하는 학습 모듈로 볼 수 있습니다. 시뮬레이션에서 만든 대량 궤적에 촉각 배열을 붙이면 CGP의 학습 데이터가 됩니다.
Do as I Do (`do_as_i_do/`)	사람 시연을 로봇 궤적으로 바꾸는 쪽입니다. CGP는 텔레옵(RelaxedIK)으로 직접 시연을 모으므로 리타겟팅 품질 문제는 텔레옵 단계에서 사람이 보정합니다.

4. 논문이 다루지 않았지만 관련 있는 연구 (내 판단)

주제	관련성	근거 수준
임피던스·강성 제어의 고전 관계 $\tau_{ext} = K(a - x)$	M_φ가 학습하는 사상의 물리적 구조입니다. 관절 PD라면 $a = x + Kp^{-1}(g - J^T f)$입니다(`03` §1). 저자들이 향후 과제로 든 "제어기 이득 조건화"의 해석식이기도 합니다.	교과서 수준의 관계. 유도는 `03`
Feel the Force (사람 시연에서 접촉력을 예측해 힘 제어로 실행하는 계열)	"예측한 힘을 제어기로 실현한다"는 같은 문제의식이고, 그리퍼 중심입니다.	내 사전 지식. 서지 확인 필요
Real-Time Chunking 등 청크 경계 처리	CGP는 8스텝 청크 동안 관측 촉각을 쓰지 않습니다(`01` §4.2). 청크 안에서 촉각 오차를 반영하는 방법과 연결됩니다.	내 해석
Tactile beyond pixels [10] (Meta 계열 다감각 촉각 표현)	같은 Meta 생태계의 사전학습 촉각 표현입니다. CGP는 촉각 인코더를 과제마다 처음부터 학습하므로, 사전학습 인코더로 바꾸는 실험이 자연스럽습니다.	인용은 됨. 결합은 내 해석

한계: 4절의 서지 정보 일부는 내 사전 지식에 근거합니다. 블로그나 git에 올리기 전에 원문을 직접 확인해야 합니다.